

Technical Documentation: LABAN Multi-Label Intent Classification

Table of Contents

- 1. [LABAN Origin](#)
- 2. [What Vanilla LABAN Is Missing for This Project](#)
- 3. [Step-by-Step: Adjusting LABAN for This Project](#)
- 4. [How to Run LABAN in This Project](#)
- 5. [LABAN Workflow in This Project](#)
- 6. [How to Evaluate LABAN](#)

1. LABAN Origin

1.1 Academic Reference

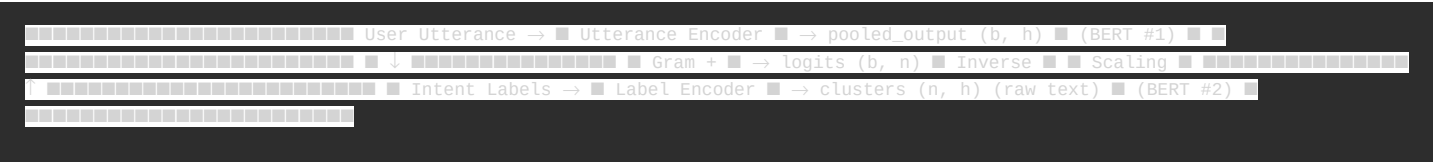
LABAN (Label-Aware BERT Attention Network) is a neural architecture for zero-shot multi-intent detection, published at EMNLP 2021:

Wu, T.-W., Su, R., & Juang, B.-H. (2021). A Label-Aware BERT Attention Network for Zero-Shot Multi-Intent Detection in Spoken Language Understanding. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*, pp. 4894-4904.

The source code is publicly available at: <https://github.com/waynewu6250/LABAN>

1.2 Core Innovation

Traditional intent classifiers use a **fixed linear head** that maps sentence embeddings to a predefined set of classes. When a new intent is added, the entire model must be retrained. LABAN solves this with a **dual-encoder architecture**:



Instead of a fixed linear layer, LABAN **projects** the utterance embedding onto the label embedding space using a Gram matrix inverse operation. This mathematical projection allows the model to classify against any set of label texts -- including labels it has never seen during training (zero-shot capability).

1.3 The Mathematical Core

The key computation in LABAN is the **linear approximation** via Gram matrix inverse:

$$\text{logits} = (U \cdot C^T) \cdot (C \cdot C^T)^{-1} \cdot \sqrt{h}$$

Where: - U = pooled utterance embedding, shape (b, h) - C = label embeddings from the label encoder, shape (n, h) - C^T = transpose of C , shape (h, n) - $(C \cdot C^T)$ = Gram matrix, shape (n, n) - h = hidden dimension size (e.g., 768) - b = batch size, n = number of labels

The Gram matrix inverse effectively performs orthogonal projection, ensuring that each logit score represents how much the utterance aligns with that specific label independent of other labels.

1.4 Original LABAN Implementation (GitHub)

The original repository (waynewu6250/LABAN) contains:

File	Purpose
model/bert_model_zsl.py	The BertZSL class with multiple surface encoder modes and label-aware layer modes
bert_laban.py	Training script for normal multi-intent detection
bert_zsl.py	Training script for zero-shot detection
config.py	Configuration class with dataset paths, hyperparameters, and mode flags
data/train_data.py	Data loading for normal detection
data/train_data_zero_shot.py	Data loading for zero-shot splits

The original model supports **6 surface encoder modes** (max-pooling, self-attentive, hierarchical, bisect, normal) and **6 label-aware layer modes** (gram, dot, dnn, student, zero-shot, normal). It was designed as a research testbed for comparing multiple architectural variants on **English SLU benchmarks** (MixATIS, MixSNIPS, FSPS, E2E, SGD).

2. What Vanilla LABAN Is Missing for This Project

The original LABAN was built as a research prototype for English SLU benchmarks. Applying it directly to the Biblical Counseling Chatbot reveals five critical gaps:

2.1 Language: English-Only Backbone

Original: Hardcoded to bert-base-uncased (English BERT).

```
Original bert_model_zsl.py, line 21 self.bert = BertModel.from_pretrained('bert-base-uncased', ...) self.bertlabelencoder = BertModel.from_pretrained('bert-base-uncased', ...)
```

Project Need: All user inputs, intent labels, and counseling conversations are in **Bahasa Indonesia**. The backbone must be an Indonesian or multilingual transformer model.

2.2 Excessive Architectural Complexity

Original: The BertZSL class includes 6 surface encoder modes, 6 label-aware layer modes, a dropout layer, a linear classifier, a mapping layer, and two relation layers -- totaling ~12 configurable components. Most of these are experimental variants tested in the paper but not needed in production.

```
Original: Unused components still loaded into memory self.dropout = nn.Dropout(0.1) self.classifier = nn.Linear(config.hidden_size, num_labels) self.mapping = nn.Linear(config.hidden_size, num_labels) self.relations1 = nn.Linear(2*config.hidden_size, 10) self.relations2 = nn.Linear(10, 1)
```

Project Need: Only the zero-shot mode with normal surface encoding is relevant. All other modes waste GPU memory and code complexity.

2.3 Data Format Mismatch

Original: Expects preprocessed .pkl (pickle) files with specific dictionary structures (intent2id_with_tokens.pkl, raw_data.pkl). Data loading is tightly coupled to the MixATIS/MixSNIPS/FSPS/E2E/SGD dataset formats.

Project Need: The chatbot uses **CSV-based datasets** (`dataset_multiintent.csv`, `dataset_multiintent_augmented.csv`) with semicolon-separated multi-label intents (e.g., "Menyatakan Perasaan Sedih dan Kehilangan; Mengisyaratkan Gejala Fisik").

2.4 No Validation/Test Pipeline

Original: The training script (`bert_laban.py`) does not include validation-based checkpointing or a formal test set. It trains for a fixed number of epochs and saves the final model.

Project Need: For a thesis-quality evaluation, the project requires an **80/10/10 train/validation/test split** with: - Per-epoch validation F1 tracking - Best-checkpoint saving based on validation F1 - Final evaluation on a held-out test set

2.5 No Numerical Stability (Gram Matrix Inversion)

Original: The Gram matrix inversion is performed directly without regularization:

```
# Original: Can crash if label embeddings are near-linearly-dependent weights = torch.mm(weights, torch.inverse(gram)) * np.sqrt(768)
```

If two label embeddings become too similar (which happens with semantically close intents like "Sedih dan Kehilangan" and "Takut dan Kecemasan"), the Gram matrix becomes singular and `torch.inverse()` produces `NaN` or crashes.

Project Need: Tikhonov regularization to ensure numerical stability.

3. Step-by-Step: Adjusting LABAN for This Project

Step 1: Replace the Backbone with an Indonesian Transformer

The English `bert-base-uncased` was replaced with `indolem/indobertweet-base-uncased`, a BERT model pre-trained on Indonesian Twitter data. The backbone is configurable via `config.py`:

```
# config.py (this project) class config: MODEL_NAME = 'indolem/indobertweet-base-uncased' hidden_size = 768
```

```
# bert_model.py (this project), lines 32-34 self.bert = AutoModel.from_pretrained(opt.MODEL_NAME, ...) self.bertlabelencoder = AutoModel.from_pretrained(opt.MODEL_NAME, ...)
```

Using `AutoModel` and `AutoTokenizer` instead of hardcoded `BertModel/BertTokenizer` ensures that the code automatically adapts to any HuggingFace model by reading its config -- enabling easy backbone swaps for evaluation (IndoBERT-Lite, DistilBERT, MiniLM, E5, etc.).

Step 2: Strip to the Essential Architecture

All experimental modes from the original BertZSL were removed. The project uses only: - **Surface Encoder:** `normal` mode (uses `pooler_output` -- the `[CLS]` token representation) - **Label-Aware Layer:** `zero-shot` mode (Gram matrix inverse projection)

The entire `BertEmbedding` class was simplified from ~227 lines (original) to **~45 lines** (this project):

```
# bert_model.py (this project) -- Complete forward() method class BertEmbedding(nn.Module): def __init__(self, num_labels=None): super().__init__() self.bert = AutoModel.from_pretrained(opt.MODEL_NAME, ...) self.bertlabelencoder = AutoModel.from_pretrained(opt.MODEL_NAME, ...) def forward(self, utterance_ids, utterance_mask, label_ids, Label_mask): # Label Encoder output_label = self.bertlabelencoder(input_ids=label_ids, ...) clusters = output_label.pooler_output # Utterance Encoder output_utterance = self.bert(input_ids=utterance_ids, ...) pooled_output = output_utterance.pooler_output # LABAN Core: Gram matrix inverse projection gram = torch.mm(clusters, clusters.permute(1,0)) gram = gram + torch.eye(gram.size(0), device=gram.device) * 1e-4 weight = torch.mm(pooled_output, clusters.permute(1,0)) logits = torch.mm(weight, torch.inverse(gram)) * np.sqrt(opt.hidden_size) return logits
```

Removed components (not needed for zero-shot mode): - `self.dropout`, `self.classifier`, `self.mapping` - `self.relations1`, `self.relations2` - `self.mode` / `self.mode2` switches - `transform()` method (5 surface encoder variants) - Pretrained weight loading (`self.pre`)

Step 3: Add Tikhonov Regularization

A small identity matrix (ϵI , where $\epsilon = 10^{-4}$) was added to the Gram matrix before inversion to prevent numerical instability:

```
# bert_model.py, line 66 gram = gram + torch.eye(gram.size(0), device=gram.device) * 1e-4
```

This is a standard regularization technique (Tikhonov regularization / ridge regression) that ensures the Gram matrix is always invertible, even when label embeddings are near-linearly-dependent.

Step 4: Build Indonesian Multi-Label Data Pipeline

A custom `LABANDataset` (PyTorch Dataset) and data preparation module were built in `data/data_preparation.py`:

```
# data/data_preparation.py # 1. Load CSV with semicolon-separated intents dsqi = pd.read_csv('data/augmentation/dataset_multiintent_augmented.csv') dsqi['intent_list'] = dsqi['Intent'].apply(lambda x: x.split('; ')) # 2. Multi-hot encoding via sklearn mlb = MultiLabelBinarizer() encoded_labels = mlb.fit_transform(dsqi['intent_list']) listof_intent = list(mlb.classes_) # 10 intent labels # 3. Tokenize the raw intent label text for the Label Encoder tokenized_intent = tokenizer(listof_intent, padding=True, truncation=True, ...) # 4. Split: 80% train, 10% validation, 10% test df_train, df_temp = train_test_split(dsqi_encoded, test_size=0.2, random_state=42) df_val, df_test = train_test_split(df_temp, test_size=0.5, random_state=42)
```

Step 5: Build Training Pipeline with Validation Checkpointing

The training script (`run_trainer.py`) was built with thesis-grade rigor:

```
# run_trainer.py -- Training loop structure optimizer = AdamW(model.parameters(), weight_decay=0.01, lr=2e-5) criterion = BCEWithLogitsLoss(reduction='sum') for epoch in range(50): # Training phase model.train() for batch in train_dataloader: logits = model(utterance_ids, utterance_mask, intent_ids, intent_mask) loss = criterion(logits, labels) loss.backward() optimizer.step() # Validation phase model.eval() val_f1 = compute_f1(val_predictions, val_targets) # Save best checkpoint if val_f1 > best_val_f1: best_val_f1 = val_f1 torch.save(model.state_dict(), 'checkpoint/IndoBERT_multi_label_zsl.pt') # Test phase (using best checkpoint) model.load_state_dict(torch.load('checkpoint/IndoBERT_multi_label_zsl.pt')) test_f1 = compute_f1(test_predictions, test_targets)
```

Step 6: Build the Inference (Predictor) Class

A predictor class in `models/multilabel/predict.py` loads the trained checkpoint and provides real-time intent classification:

```
# predict.py -- Inference pipeline class predictor: def __init__(self, model_path, threshold=0.5): self.model = BertEmbedding(num_labels=10) self.model.load_state_dict(torch.load(model_path)) self.model.eval() def predict(self, text): logits = self.model(utterance_ids, utterance_mask, intent_ids, intent_mask) probz = torch.sigmoid(logits).squeeze().cpu().numpy() detected_intents = [] for i, label in enumerate(self.label_names): if probz[i] >= self.threshold: detected_intents.append(label) return { "pertanyaan": text, "inten_terdeteksi": detected_intents, "apakah multilabel": len(detected_intents) > 1, "skore": scores }
```

Summary: Original vs. Project LABAN

Aspect	Original LABAN (GitHub)	This Project
Language	English (bert-base-uncased)	Indonesian (indolem/indobertweet-base-uncased)

Backbone	Hardcoded BERT	Configurable via <code>AutoModel + config.py</code>
Architecture	6 surface modes, 6 label modes	Only <code>normal + zero-shot</code>
Extra Layers	Dropout, classifier, mapping, relations	None (pure zero-shot projection)
Numerical Stability	No Gram regularization	Tikhonov regularization ($\epsilon = 10^{-4}$)
Data Format	Pickle files (<code>.pkl</code>)	CSV with semicolon-separated multi-labels
Data Split	No formal validation	80/10/10 train/val/test
Checkpointing	Save final model	Save best-validation-F1 model
Evaluation	Manual test mode	Automated F1 + per-intent classification report
Datasets	MixATIS, MixSNIPS, FSPS, E2E, SGD	Custom Indonesian counseling dataset
Labels	18-26 English SLU intents	10 Indonesian emotional intents
Code Size	~227 lines (model only)	~45 lines (model only)

4. How to Run LABAN in This Project

4.1 Prerequisites

1. **Virtual environment activated:** `cbenv\Scripts\activate`
2. **Dependencies installed:** `transformers, torch, scikit-learn, pandas, matplotlib`
3. **Dataset available:** `data/augmentation/dataset_multiintent_augmented.csv` (or `data/dataset_multiintent.csv` as fallback)
4. **GPU recommended:** CUDA-enabled GPU (CPU works but training is slow)

4.2 Training the Model

```
cbenv\Scripts\activate python models/multilabel/run_trainer.py
```

This runs the full training pipeline: 1. Loads the augmented dataset 2. Splits into 80/10/10 train/val/test 3. Trains for 50 epochs with AdamW optimizer 4. Saves best checkpoint to `checkpoint/IndoBERT_multi_label_zsl.pt` 5. Evaluates on the test set 6. Produces training metrics PNG

4.3 Running Interactive Prediction

```
python models/multilabel/predict.py
```

This starts an interactive CLI where you can type counseling utterances and see the detected intents:

```
Pasien : Saya merasa sangat sedih dan takut setelah kehilangan pekerjaan Intent terdeteksi : ['Menyatakan Perasaan Sedih dan Kehilangan', 'Menyatakan Perasaan Takut dan Kecemasan'] Apakah multilabel? : True Skor : {'Menyatakan Perasaan Sedih dan Kehilangan': 0.8723, ...}
```

4.4 Configuration

All hyperparameters are centralized in `config.py`:

Parameter	Value	Description
MODEL_NAME	indolem/indoberttweet-base-uncased	HuggingFace transformer backbone
hidden_size	768	Embedding dimension (must match backbone)
max_len	50	Maximum token sequence length
BATCH_SIZE	16	Training batch size
epochs	50	Maximum training epochs
LEARNING_RATE	2e-5	AdamW learning rate
threshold	0.5	Sigmoid threshold for multi-label prediction
EMBED_MODEL	paraphrase-multilingual-MiniLM-L12-v2	Embedding model for RAG (separate from classifier)

5. LABAN Workflow in This Project

5.1 End-to-End System Workflow

LABAN operates as the **intent classification component** within the larger RAG chatbot pipeline:

```
User Input (Browser) ↓ POST /chat app.py (Flask) ↓ SessionManager.chat() ■■■ Stage tracking (6 stages) ■■■ Transition signal
Detection ↓ RAGEngine.generate_response() ■ ■■■ [1] LABAN Classification (predict.py) ■■■ THIS DOCUMENT ■ Input: "Saya merasa
sangat sedih" ■ Output: ["Sedih dan Kehilangan"] + scores ■ ■■■ [2] QnA FAISS Search (vector_db.py) ■ Uses: user_input →
Similar QnA pair ■ ■■■ [3] Bible FAISS Search (vector_db.py) ■ Uses: detected intents + user_input → Bible verse ■ Active
only in: solusi, relaksasi stages ■ ■■■ [4] Prompt Assembly (LangChain PromptTemplate) ■ Combines: intents + QnA answer +
Bible verse + stage instruction ■ ■■■ [5] Gemini 2.5-Flash API Call Output: Counselor response
```

5.2 LABAN's Internal Data Flow (Forward Pass)

When `predictor.predict("Saya merasa sangat sedih")` is called:

```
Step 1: Tokenize the user input "Saya merasa sangat sedih" → input_ids: [2, 1045, 7903, 12345, 8765, 3] (token IDs) →
attention_mask: [1, 1, 1, 1, 1, 1] Step 2: Tokenize all 10 intent labels (done once at startup) ["Mengisyaratkan Butuh Bantuan
Profesional", ..., "Menyatakan Reaksi Terkejut"] → intent_ids: shape (10, max_label_len) → intent_mask: shape (10,
max_label_len) Step 3: Utterance Encoder (BERT #1) input_ids → IndoBERTweet → pooled_output: shape (1, 768) Step 4: Label
Encoder (BERT #2) intent_ids → IndoBERTweet → clusters: shape (10, 768) Step 5: Gram Matrix Projection gram = clusters @
clusters.T → shape (10, 10) gram = gram + 0.0001 * I → Tikhonov regularization weight = pooled_output @ clusters.T → shape (1,
10) logits = weight @ inverse(gram) * sqrt(768) → shape (1, 10) Step 6: Sigmoid + Threshold probabilities = sigmoid(logits) →
[0.12, 0.03, ..., 0.87, 0.45, ...] detected = [label for prob > 0.5] → ["Sedih dan Kehilangan"]
```

5.3 Multi-Label Behavior

Unlike single-label classifiers that pick the single highest-scoring class, LABAN applies the sigmoid function independently to each logit. Any label whose sigmoid probability exceeds the threshold (0.5) is detected. This enables multi-label output:

```
Input: "Aku takut dan sangat sedih karena kehilangan pekerjaanku" Probabilities: Sedih dan Kehilangan: 0.87 ✓ (above 0.5) Takut dan Kecemasan: 0.72 ✓ (above 0.5) Gejala Fisik: 0.15 ✗ Marah dan Frustrasi: 0.31 ✗ ... Result: Multi-label = ["Sedih dan Kehilangan", "Takut dan Kecemasan"]
```

5.4 Stage-Conditional Behavior

LABAN classification is **skipped** in two counseling stages: - **pembukaan** (opening): The chatbot greets the user; no classification needed. - **bantuan_profesional** (professional help): A safety-net branch stage triggered by keywords.

In all other stages, LABAN classifies every user message. The detected intents are accumulated across turns by the `SessionManager` and used to drive Bible verse selection during the `solusi` and `relaksasi` stages.

6. How to Evaluate LABAN

Three evaluation scripts exist in the `evaluation/` directory, each measuring a different aspect of LABAN's performance.

6.1 Backbone Comparison (`compare_embed_models.py`)

Purpose: Compare 6+ transformer backbones to determine which pre-trained model produces the best F1 score when plugged into the LABAN architecture.

Command:

```
python evaluation/compare_embed_models.py
```

What It Does: 1. Iterates over a list of backbone models (IndoBERTweet, IndoBERT-Lite, IndoBERT, MiniLM, Multilingual-E5-small, DistilBERT-multilingual) 2. For each backbone: instantiates a fresh LABAN model, trains for 50 epochs with identical hyperparameters (same seed, LR, batch size) 3. Records best-validation-F1 checkpoint per backbone 4. Evaluates each on the same held-out test set 5. Produces per-intent classification reports

Output: - `evaluation/results/backbone_comparison_summary.csv` -- Overall F1 per backbone - `evaluation/results/backbone_comparison_per_intent.csv` -- Per-intent F1 breakdown - `evaluation/results/<model_tag>_training_curve.png` -- Loss + F1 curves per backbone

What to Report in Thesis: A comparison table showing Test F1-Micro, F1-Macro, Precision, and Recall for each backbone, justifying the selection of `indolem/indobertweet-base-uncased` as the production model.

6.2 Zero-Shot Seen/Unseen Evaluation (`eval_seen_unseen.py`)

Purpose: Measure LABAN's zero-shot generalization -- its ability to classify intents it has never seen during training.

Command:

```
python evaluation/eval_seen_unseen.py
```

What It Does: 1. Defines 3 different seen/unseen splits (each holds out 3 of 10 intents as "unseen") 2. For each split: - Trains LABAN with unseen intent columns **masked** (zeroed out) in the training data - The Label Encoder still receives ALL 10 intent label texts (key for zero-shot) - Evaluates on the full test set with ALL 10 columns active 3. Computes F1-Macro for Seen, Unseen, and All intents

Output: - `evaluation/results/seen_unseen_summary.csv` -- Per-split F1 scores - `evaluation/results/seen_unseen_per_intent.csv` -- Per-intent per-split breakdown - `evaluation/results/seen_unseen_aggregate.csv` -- Mean and std across all splits

What to Report in Thesis: The aggregate F1-Macro Unseen score demonstrates LABAN's zero-shot capability. A high unseen F1 (e.g., > 0.5) proves the model can generalize to new intent categories using only the label text, validating the LABAN architecture's theoretical premise.

6.3 Cosine Similarity Heatmap (eval_cosine_heatmap.py)

Purpose: Visualize the quality of the learned embedding space by measuring cosine similarity between utterance embeddings and label embeddings.

Command:

```
python evaluation/eval_cosine_heatmap.py
```

What It Does: 1. Loads the trained LABAN checkpoint 2. Extracts embeddings from both encoders on the test set 3. Produces three heatmaps: - **Label-vs-Label:** How distinct are the 10 intent labels from each other? - **Utterance Centroid-vs-Label:** Do average utterance embeddings align with their target labels? - **Per-Sample Utterance-vs-Label:** How consistent is the alignment across individual samples?

Output: - evaluation/results/cosine_label_vs_label.png - evaluation/results/cosine_centroid_vs_label.png - evaluation/results/cosine_utterance_vs_label.png - evaluation/results/cosine_similarity_metrics.csv

What to Report in Thesis: The Label Separation Gap and Centroid Alignment Gap metrics quantify how well the dual-encoder architecture separates different intents. High gaps confirm that the model's embedding space is well-structured for multi-label classification.

6.4 Summary of LABAN Evaluation

Evaluation	Script	What It Measures	Key Metric
Backbone Comparison	compare_embed_models.py	Which pre-trained model works best with LABAN?	Test F1 per backbone
Zero-Shot Seen/Unseen	eval_seen_unseen.py	Can LABAN classify intents it has never been trained on?	F1-Macro Unseen
Cosine Similarity Heatmap	eval_cosine_heatmap.py	Is the learned embedding space well-structured?	Label Separation Gap, Alignment Gap